

Activation Functions and Loss Functions

Contents

- ✓ Introduction to Activation functions
- ✓ Types of Activation functions
- ✓ Loss function and its types
- ✓ Designing a Neural Network

NEURAL NETWORKS

Activation Function

- ✓ In feedforward propagation, the activation function is a mathematical function that is applied to a neurons output, and it decides whether that output should be passed to the next layer or not.
- ✓ It transforms the summed weighted input from the node into an output value to be fed to the next hidden layer or as output.
- ✓ They are the backbone of neural networks enabling them to capture non-linear relationships in data.

Without nonlinearity, a neural network would only function as a simple linear regression model.

Input Layer

Hidden Layer

Output Layer

Weighted sum of inputs:

$$z_1 = w_{11} * x_1 + w_{21} * x_2 + b_1$$

Activation function:

$$a_1 = g(z_1)$$

Activation function:

$$a_2 = g(z_2)$$

Weighted sum of inputs:

$$z_2 = w_{12} * x_1 + w_{22} * x_2 + b_1$$

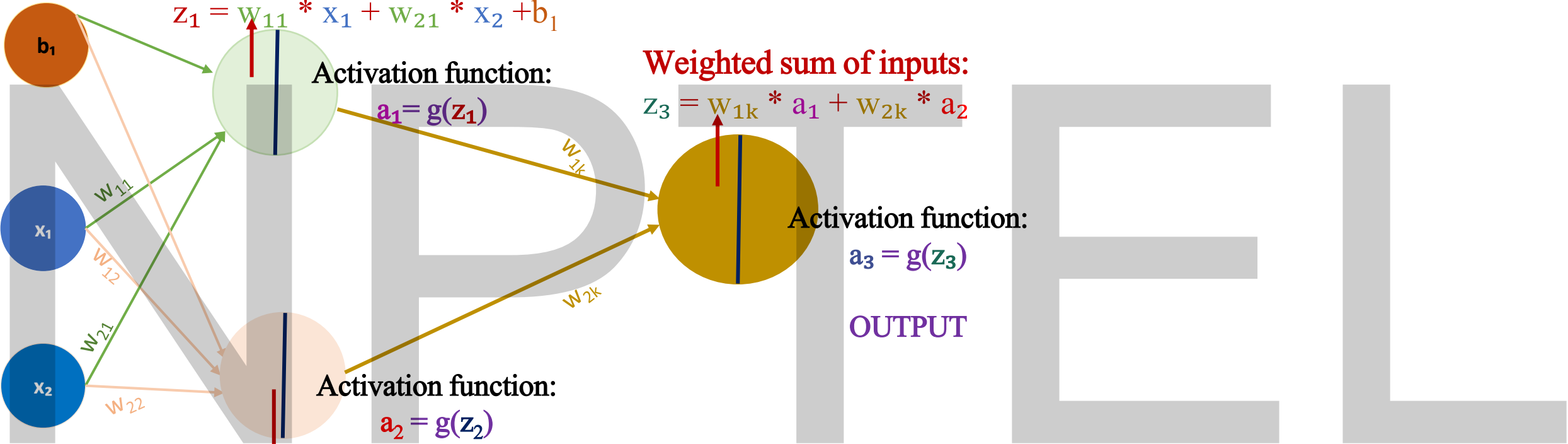
Weighted sum of inputs:

$$z_3 = w_{1k} * a_1 + w_{2k} * a_2$$

Activation function:

$$a_3 = g(z_3)$$

OUTPUT



Types of Activation functions

Two types of activation functions:

- ✓ Linear Activation Function
- ✓ Non-linear Activation Functions
 - ✓ Sigmoid
 - ✓ Tanh
 - ✓ Softmax
 - ✓ ReLU
 - ✓ Leaky ReLU

Linear Activation Function

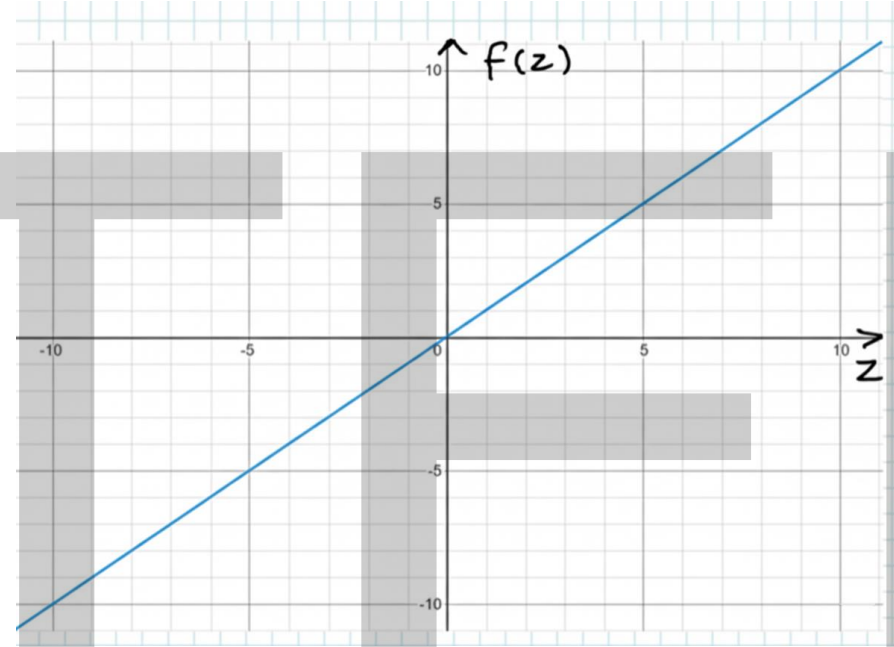
Also known as identity function

Linear function

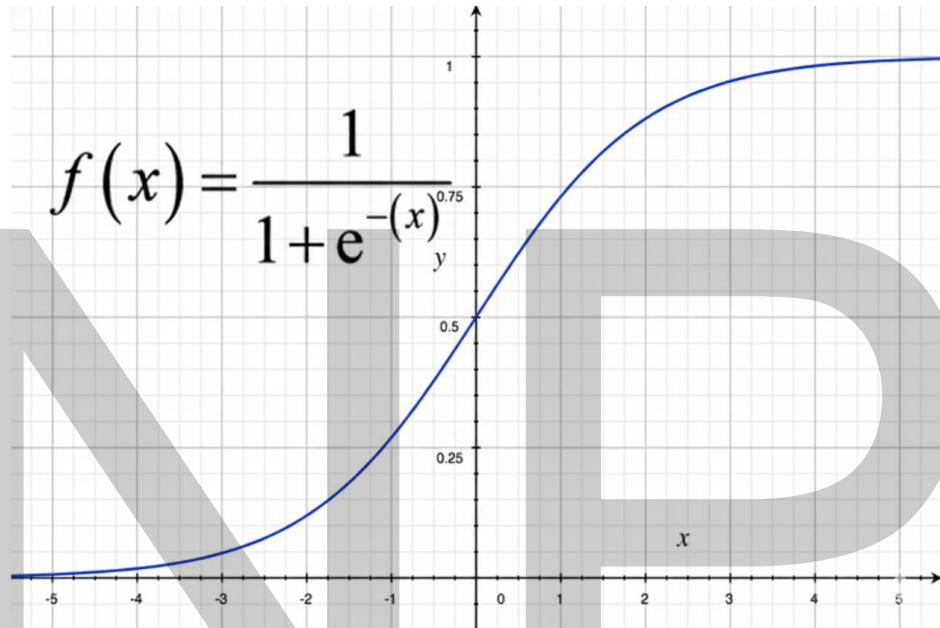
$$f(z) = z$$

Range of Possible outcomes. $(-\infty, +\infty)$

Used in: Output layer for Regression



Nonlinear Activation Function- Sigmoid activation function



Binary
Image
Classifier

Dog

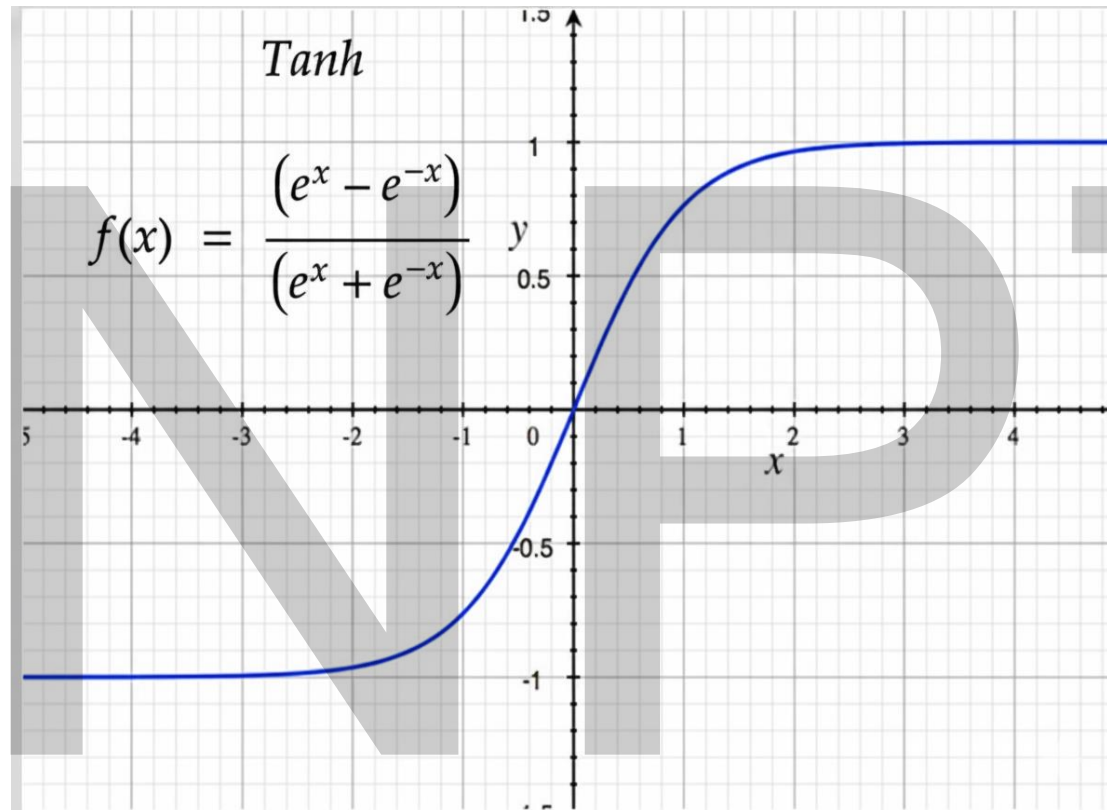
Not Dog



x	$f(x)$
-5	0.0067
-3	0.0474
-1	0.2689
0	0.5
1	0.7311
3	0.9526
5	0.9933
-20	≈ 0.00000000206
20	≈ 0.999999998

Used in the output layer for Binary Classifier

Tangent hyperbolic(*Tanh*) activation function



x	$f(x)$
-5	-0.9999
-3	-0.9950
-1	-0.7616
0	0
1	0.7616
3	0.9950
5	0.9999

Output is zero-centered

Used in hidden layers as it makes the mean of the hidden layer comes to zero or very near to zero

➡ Helps in learning in the subsequent layers

Softmax Activation Function

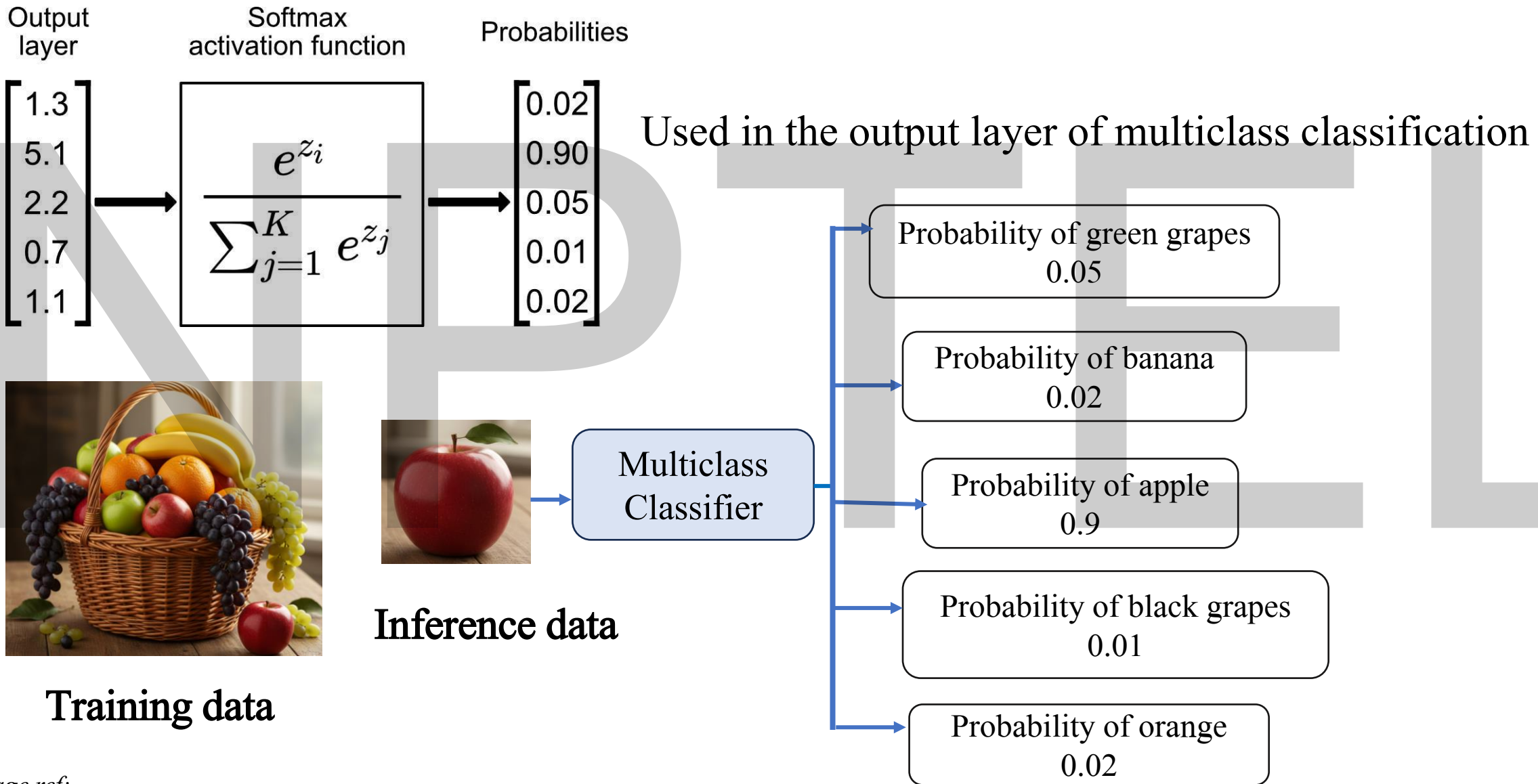
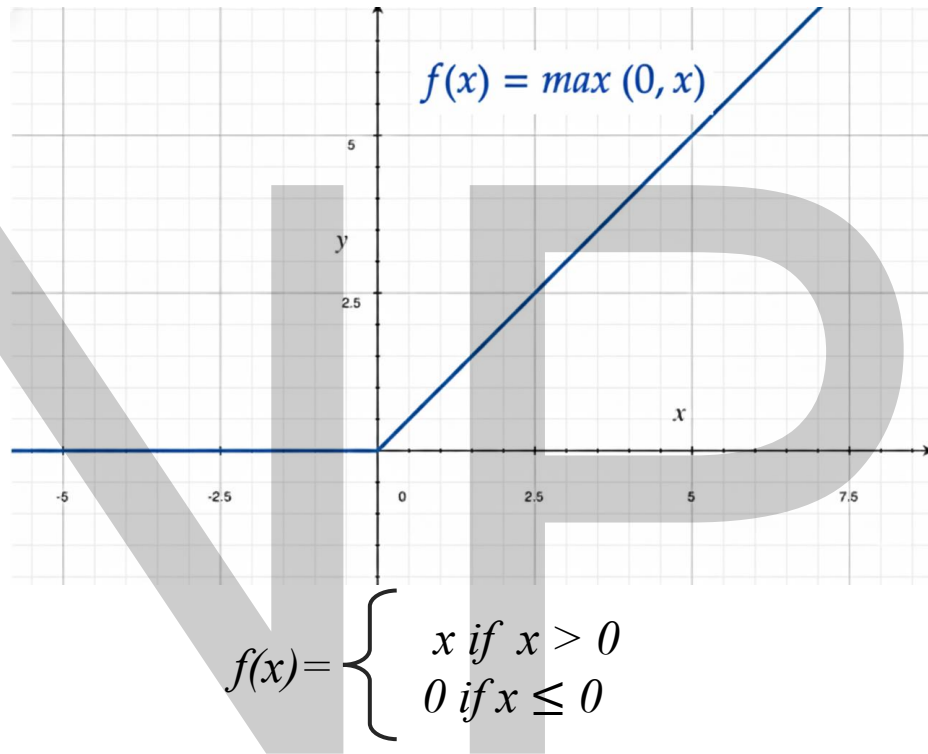


Image ref:

<https://towardsdatascience.com/softmax-activation-function-explained-a7e1bc3ad60/>

ReLU (Rectified Linear Unit) Activation function



NOTE

Default activation function for many types of neural networks

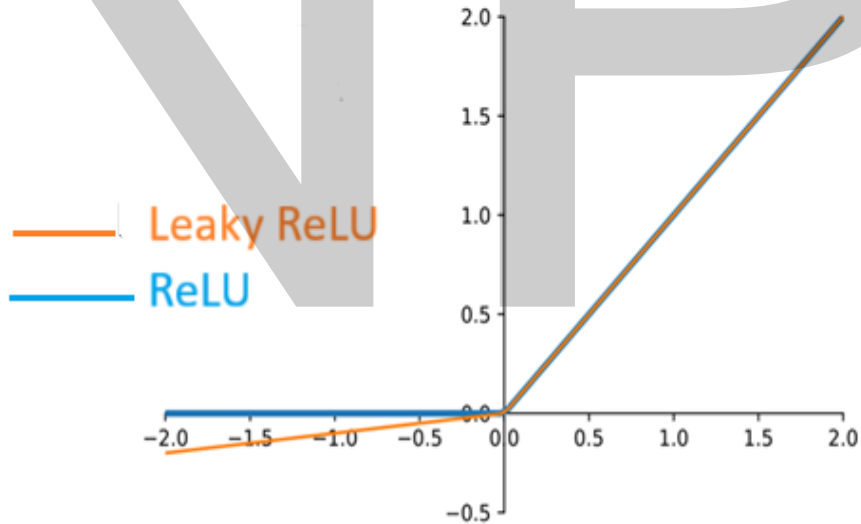
- Computationally efficient as it does a thresholding operation.
- It introduces sparsity in the network as only a fraction of neurons activate at any point of time.
- It helps in backpropagation as its derivative is either 0 or 1.

Leaky ReLU

Drawback of ReLU Activation function:

Dying ReLU Problem:

- Neurons can sometimes become inactive and only output 0.
- This happens when large negative inputs result in zero gradient, leading to neurons that never activate and cannot learn further.



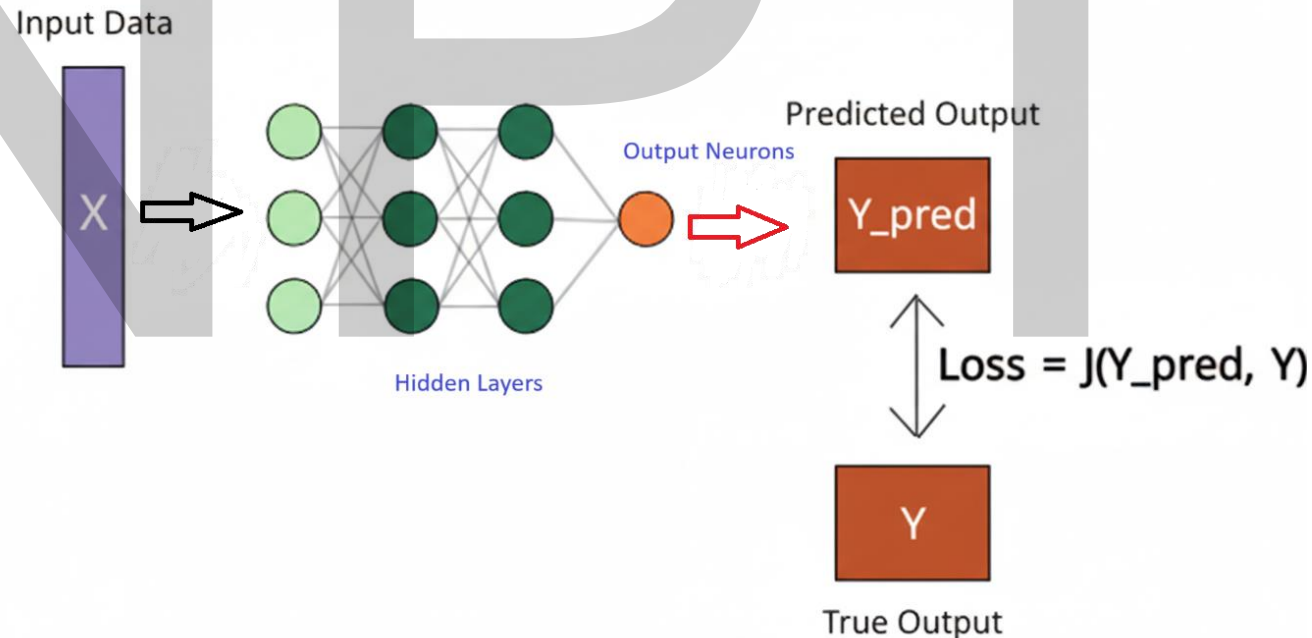
$$\text{Leaky ReLU}(x) = \begin{cases} x, & \text{if } x > 0 \\ 0.01 \cdot x & \text{if } x \leq 0 \end{cases}$$

Loss Functions

Loss Functions

Loss function measures how well a machine learning model's predicted outputs match the true output labels.

The loss function is used to **optimize the model** by minimizing the loss, which means that the model makes fewer mistakes on the training data.



Loss function for Regression Problem

✓ Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

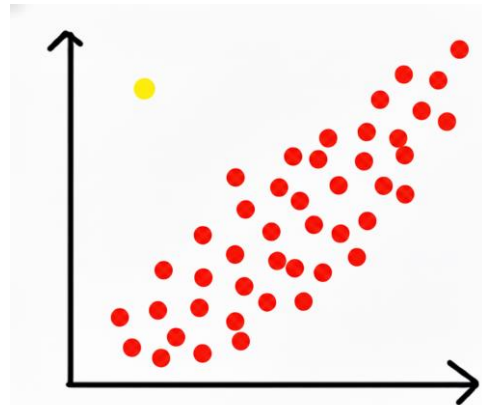
y_i = Actual value
 $\hat{y}_i$ = Predicted value
 n = number of observations

Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Why are errors either squared or have absolute value in Machine Learning?

MSE is sensitive to outliers



$$\text{Error} : y_i - \hat{y}_i$$

When error is small: 0.1, 0.5, 0.3

MSE

$$\begin{aligned} &= \frac{1}{3} [(0.1)^2 + (0.5)^2 + (0.3)^2] \\ &= \frac{1}{3} [0.01 + 0.25 + 0.09] \\ &= \frac{1}{3} [0.35] = 0.116 \end{aligned}$$

When error is large : errors:- 1, 5, 3

$$\begin{aligned} \text{MSE} &= \frac{1}{3} [1^2 + 5^2 + 3^2] \\ &= \frac{1}{3} [1 + 25 + 9] \\ &= \frac{1}{3} [35] = 11.67 \end{aligned}$$

MAE

$$\begin{aligned} &\frac{1}{3} [0.1 + 0.5 + 0.3] \\ &= \frac{1}{3} [0.9] = 0.3 \end{aligned}$$

$$\text{MAE} > \text{MSE}$$

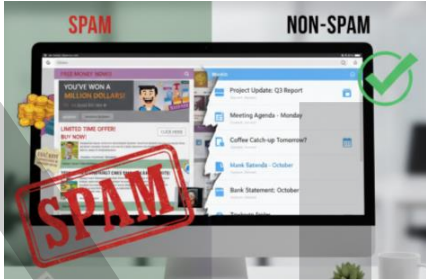
MAE

$$\begin{aligned} &\frac{1}{3} [1 + 5 + 3] \\ &= \frac{1}{3} [9] = 3 \end{aligned}$$

$$\text{MSE} > \text{MAE}$$

Loss Functions for Classification Problem

Binary Classification



Spam
Not Spam

Binary Cross Entropy Loss

Number of output nodes = 1

Multiclass Classification



✓ Horse
Rabbit
Dog

Categorical Cross Entropy Loss

Number of output nodes
= number of classes

Multilabel Classification



Horse
✓ Cat
Dog
✓ Bird

Binary cross entropy Loss
for each label

Number of output nodes
= number of labels

Binary Cross Entropy Loss (Log Loss)

Binary Classification uses sigmoid activation function



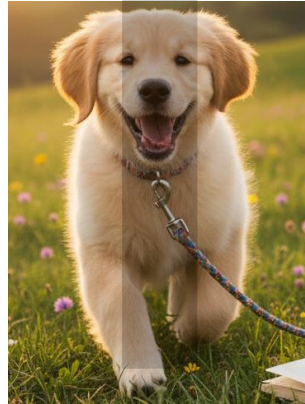
Sigmoid activation function squish any real value between values 0 and 1.

Output node Value > 0.5 → Positive class (1)

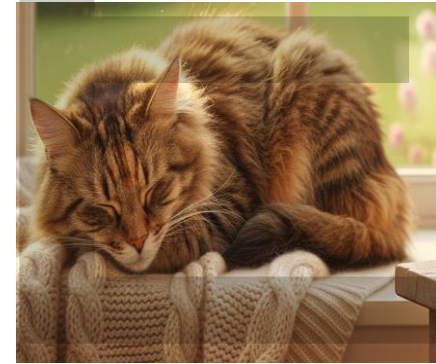
Output node Value < 0.5 → Negative Class (0)

$$Loss = (Y)(-\log(Y_{pred})) + (1 - Y)(-\log(1 - Y_{pred}))$$

Positive class (true class)

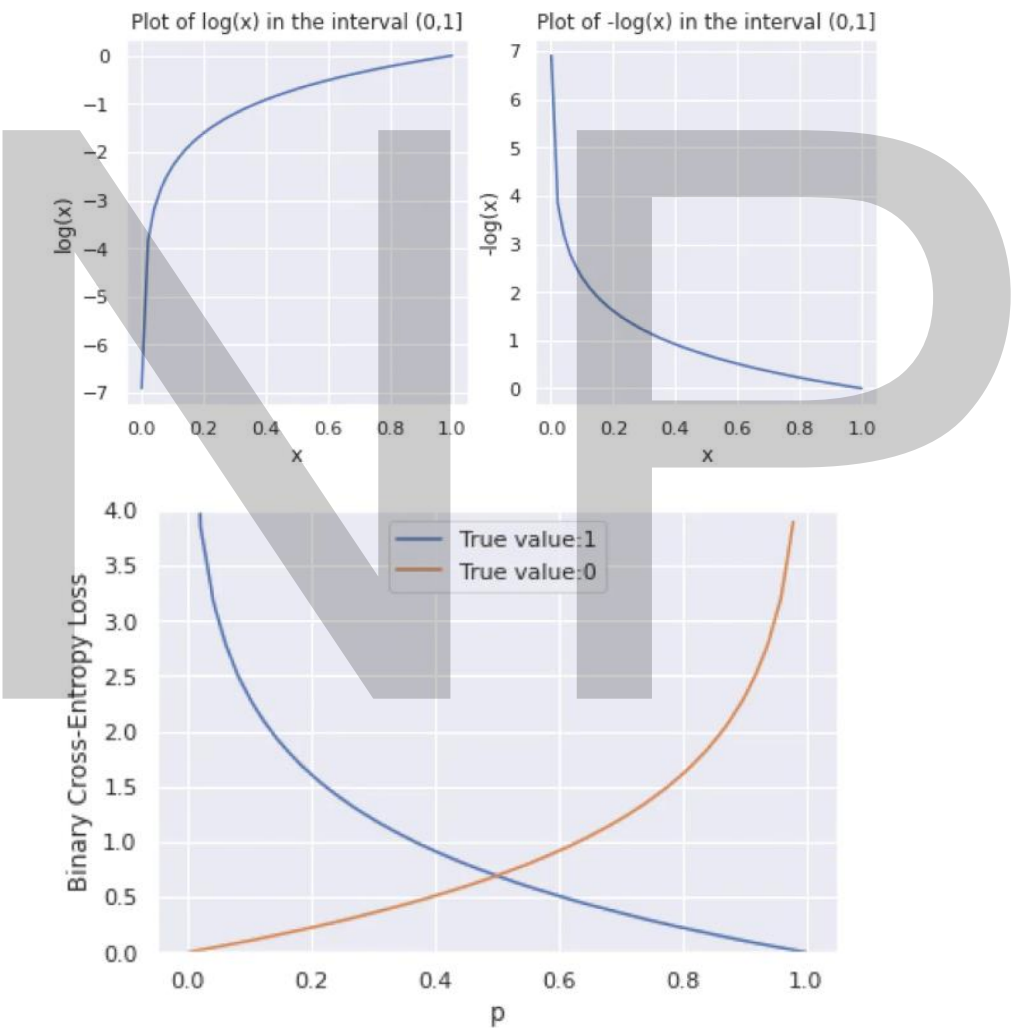


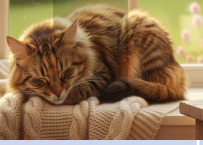
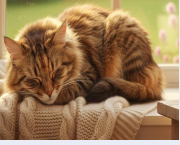
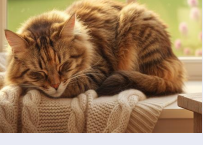
Negative class



For a single instance

$Loss = (Y)(-log(Y_{pred})) + (1 - Y)(-log(1 - Y_{pred}))$



Y (true class)	Y pred (Predicted Class)	Loss
1	1	0
		
1	0	High loss
		
0	1	High loss
		
0	0	0
		

Example of Binary Cross Entropy Loss

Consider the given actual output and the predicted output for 4 samples are given in the table below

$$\begin{aligned} \text{BCE} &= -\frac{1}{m} \sum (y_i \log(y'_i) + (1 - y_i) \log(1 - y'_i)) = \\ &= \frac{[1 \cdot \log(0.8) + (1 - 1) \log(1 - 0.8) + 0 \cdot \log(0.2) + (1 - 0) \log(1 - 0.2) \\ &\quad + 0 \cdot \log(0.6) + (1 - 0) \log(1 - 0.6) + 1 \cdot \log(0.9) + (1 - 1) \log(1 - 0.9)]}{4} \\ &\approx 0.367 \end{aligned}$$

Samples	Actual Output (y)	Predicted Output (y')
S ₁	1	0.8
S ₂	0	0.2
S ₃	0	0.6
S ₄	1	0.9

Loss function for Multiclass classification

Categorical Cross Entropy Loss

No of output nodes = number of classes

Uses Softmax Activation function

Softmax ensures

- all the output nodes have values between 0–1
- sum of all output node values always equals to 1

$\begin{bmatrix} 1.3 \\ 5.1 \\ 2.2 \\ 0.7 \\ 1.1 \end{bmatrix}$

Softmax

$\begin{bmatrix} 0.02 \\ 0.90 \\ 0.05 \\ 0.01 \\ 0.02 \end{bmatrix}$

Predicted class

Categorical Cross Entropy Loss

$$\text{Loss} = - \sum_{i=1}^C (y_i * \log \hat{y}_i)$$

Also called as Softmax Loss

C=No. of Classes

y_i = true label (one-hot encoded, i.e., only one of them is 1, others are 0)

$\hat{y}_i$ = predicted probability for class i (from the softmax output)

One hot Encoding

id	Animal
1	Dog
2	Cat
3	Rabbit
4	Bird

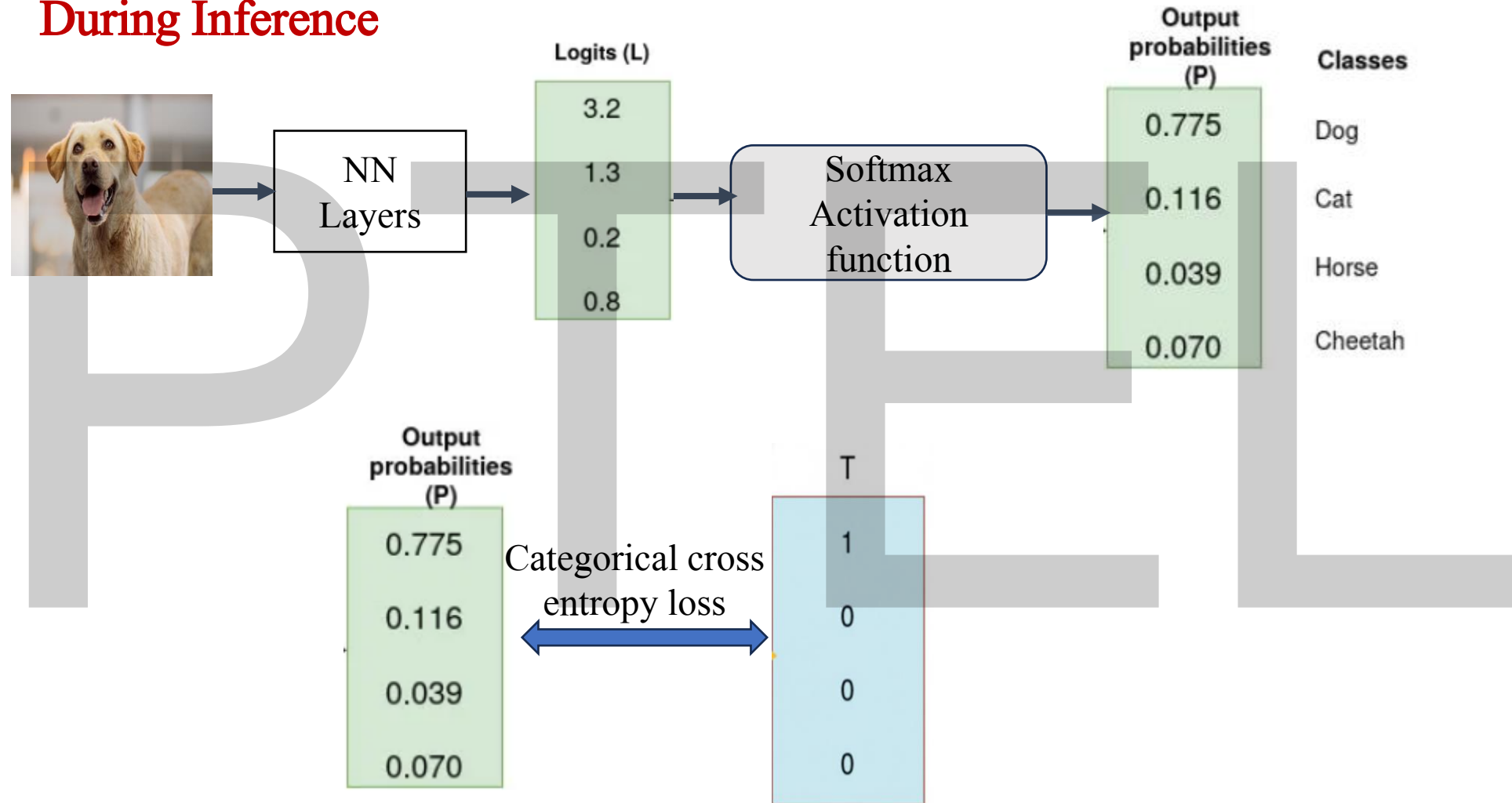


id	Animal-Dog	Animal-Cat	Animal-Rabbit	Animal-bird
1	1	0	0	0
2	0	1	0	0
3	0	0	1	0
4	0	0	0	1



Training dataset

During Inference



Example of Categorical Cross Entropy

If there are 3 classes and the true class is class 2:

True label: $y = [0, 1, 0]$

Prediction: $\hat{y} = [0.1, 0.7, 0.2]$

$$\text{Loss} = -\sum_{i=1}^c (y_i * \log \hat{y}_i)$$

$$\text{Loss} = -(0 * \log(0.1) + 1 * \log(0.7) + 0 * \log(0.2)) = -\log(0.7) = 0.1549$$

Loss function for Multilabel classification

- For multi-label classification with multiple classes, the binary cross-entropy loss is computed for each class independently and then summed over all classes to obtain the total loss.

Use Sigmoid Activation for Each Output Node:



Use Binary Cross-Entropy Loss for each class

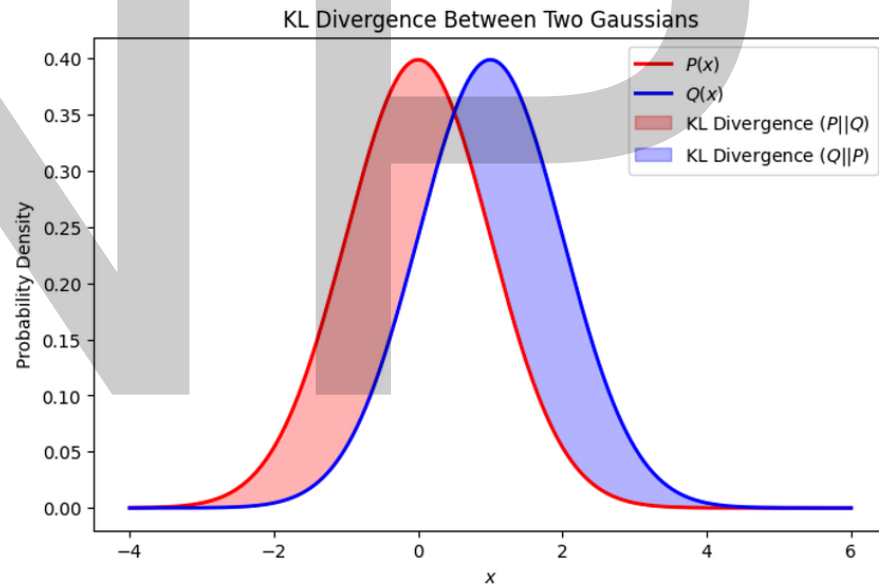


sum over all classes to obtain the total loss.



Kullback Leibler Divergence Loss (KL Divergence Loss)

- In machine learning, especially in models involving neural networks, KL Divergence is often applied in tasks where the model outputs are probabilities (ex: generative models)
- By minimizing KL Divergence, the model ensures that its predicted probability distribution closely aligns with the true distribution



Kullback Leibler Divergence Loss (KL Divergence Loss)

KL Divergence formula :

$$D_{KL}(P \parallel Q) = \sum_i P(i) \log\left(\frac{P(i)}{Q(i)}\right) \quad \text{for discrete values of } i$$

$$\int_i P(i) \log\left(\frac{P(i)}{Q(i)}\right) \quad \text{for continuous values of } i$$

Where

$D_{KL}(P \parallel Q)$ is the Kullback-Leibler divergence between two probability distributions P and Q

$P(i)$ is the probability of event i under distribution P

$Q(i)$ is the probability of event i under distribution Q

$$D_{KL}(P \parallel Q) \geq 0$$

$$D_{KL}(P \parallel Q) = 0 \text{ only if } P=Q$$

$$D_{KL}(P \parallel Q) \neq D_{KL}(Q \parallel P)$$

Designing a Neural Network Architecture

Dataset

Input layer

Number of input neurons
= No of features in the dataset

Hidden layers

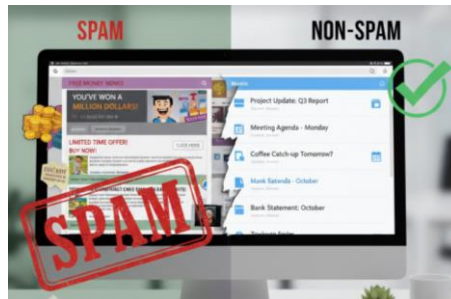
Few layers for simple task

Many layers for complex
high-dimensional data.

H	I	J	K	L
Sepal Length	Sepal Width	Petal Length	Petal Width	Class
5.1	3.5	1.4	0.2	setosa
4.9	3	1.4	0.2	setosa
7	3.2	4.7	1.4	versicolor
6.4	3.2	4.5	1.5	versicolor
6.9	3.1	4.9	1.5	versicolor
6.6	3	4.4	1.4	versicolor
4.7	3.2	1.3	0.2	setosa
5.1	3.5	1.4	0.3	setosa
5.2	3.5	1.5	0.2	setosa
6.3	2.9	5.6	1.8	virginica
6.5	3	5.8	2.2	virginica
7.6	3	6.6	2.1	virginica
5.2	3.4	1.4	0.2	setosa
4.7	3.2	1.6	0.2	setosa
4.8	3.1	1.6	0.2	setosa
5.4	3.4	1.5	0.4	setosa
6.8	2.8	4.8	1.4	versicolor

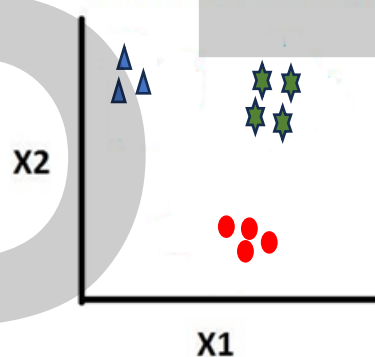
Output layer

1 neuron for binary
classification



No of neurons= number
of classes for multiclass
classification

Multi-class Classification



No of neurons= number of
labels for multilabel
classification



Label
[1 1 1]

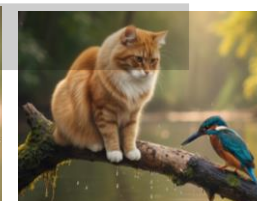
C=3
✓ Cat
✓ Bird
✓ Flower



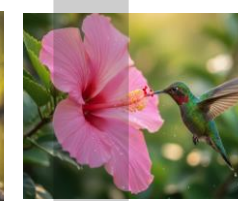
Label
[1 0 0]



Label
[0 1 0]



Label
[1 1 0]



Label
[0 1 1]

Designing a Neural Network for real world problems

Application	Output Neurons	Activation (Output)	Loss Function
Handwritten Digit Recognition (MNIST)	10 (digits 0–9)	Softmax	Categorical Cross entropy
Spam Email Detection	1	Sigmoid	Binary Cross entropy
Image Tagging (Cat/Dog/Bird can co-exist)	3 (one per label)	Sigmoid	Binary Cross entropy
Stock Price Prediction	1	Linear	Mean Squared error

Reference Books:

- ✓ Bishop, C. M., & Bishop, H. (2023). Deep learning: Foundations and concepts. Springer Nature.
- ✓ Ian Goodfellow, Yoshua Bengio, and Aaron Courville, Deep Learning, MIT Press, 2016.

NPTREL

Summary

Next Session

Optimization of Machine Learning Algorithms

NPTREL

Thank you